TOMATO VISION

Tài liệu yêu cầu và phạm vi phát triển Frontend
Phiên bản định hướng: MVP desktop application

Mục tiêu: xây dựng ứng dụng desktop kiểm tra chất lượng cà chua bằng thị giác máy tính. Ứng dụng cung cấp giao diện vận hành, hiển thị camera và kết quả YOLO, thống kê, lưu lịch sử và cấu hình hệ thống. Frontend không tự triển khai thuật toán nhận diện; nó giao tiếp với các backend service hiện có.

1. Công nghệ và nguyên tắc kiến trúc

Thành phần	Công nghệ / yêu cầu
Desktop GUI	Python + PySide6 (Qt). Ưu tiên ứng dụng desktop, không cần React/HTML/CSS/JavaScript.
AI inference	ONNX Runtime; tái sử dụng model ONNX và inference/postprocessing code đang có.
Camera / ảnh	OpenCV + NumPy; hỗ trợ webcam trước, mở rộng camera công nghiệp sau.
Database	SQLite cho phiên bản chạy độc lập trên một máy.
Đa nền tảng	Windows và Linux; đóng gói và kiểm thử riêng trên từng hệ điều hành.

Nguyên tắc kiến trúc

• Giao diện chỉ hiển thị dữ liệu và nhận thao tác người dùng; không chứa trực tiếp logic YOLO hoặc điều khiển camera.

• Frontend gọi các service như CameraService, DetectionService, ResultService và ConfigurationService.

• Không chạy vòng lặp camera/inference trên UI thread. Dùng QThread và Signal/Slot để tránh treo giao diện.

• Giữ nguyên các module backend đã chạy ổn định; tích hợp qua interface rõ ràng, không viết lại nếu chưa cần.

2. Các màn hình và chức năng bắt buộc

Màn hình	Yêu cầu chính	Mức ưu tiên
Dashboard	Tổng số sản phẩm; số Fresh/Rotten; tỷ lệ lỗi; trạng thái camera/model/database; thống kê tổng quan.	MVP
Live Inspection	Hiển thị video realtime; bounding box; nhãn Fresh/Rotten; confidence; Start/Stop; FPS và inference time.	MVP
Inspection History	Danh sách kết quả; thời gian; nhãn; confidence; tìm kiếm/lọc; xem ảnh; xuất CSV/Excel.	MVP
Settings	Chọn camera; cấu hình model; confidence threshold; độ phân giải và các tuỳ chọn vận hành.	MVP
Login / User Management	Đăng nhập/đăng xuất; quản lý tài khoản và vai trò Administrator, Engineer, Operator.	Giai đoạn sau MVP
System Monitoring	CPU/RAM/GPU nếu khả dụng; FPS; latency; trạng thái module; log và cảnh báo lỗi.	Nâng cấp

3. Yêu cầu giao diện và trải nghiệm

• Phong cách: phần mềm công nghiệp hiện đại, dark theme, bố cục rõ ràng, hạn chế hiệu ứng trang trí.

• Bố cục: sidebar cố định bên trái; header hiển thị tên trang và trạng thái; vùng nội dung chính ở giữa.

• Điều hướng: chuyển trang không làm mất trạng thái ứng dụng không cần thiết.

• Trạng thái: phân biệt rõ Ready, Running, Stopped, Disconnected và Error; không chỉ dùng màu sắc mà cần có chữ/icon.

• Khả năng sử dụng: nút Start/Stop dễ thấy; thông tin quan trọng dễ đọc; cửa sổ co giãn hợp lý.

• Camera view: giữ đúng tỷ lệ hình ảnh, không kéo méo video; lớp bounding box phải khớp với ảnh hiển thị.

4. Xử lý luồng camera và AI

• Camera worker chịu trách nhiệm lấy frame; detection worker hoặc pipeline xử lý inference độc lập với giao diện.

• Dùng Qt Signal/Slot để gửi frame, detection result, FPS, latency và trạng thái về UI.

• Ưu tiên xử lý frame mới nhất hoặc dùng queue có giới hạn để tránh độ trễ tăng dần khi inference chậm.

• Khi camera mất kết nối: thông báo rõ, thử reconnect theo chính sách cấu hình và cho phép dừng an toàn.

• Khi model chưa tải hoặc inference lỗi: không cho chạy inspection; hiển thị lỗi có thể hiểu và ghi log.

• Đóng ứng dụng phải dừng worker, giải phóng camera và tài nguyên inference một cách an toàn.

5. Lưu trữ và bảo mật

• Lưu mỗi lần kiểm tra tối thiểu: timestamp, class label, confidence, camera/source và đường dẫn ảnh nếu có lưu ảnh.

• Có cơ chế truy vấn lịch sử, lọc theo thời gian/nhãn và xuất báo cáo.

• Nếu bật tài khoản: lưu password dạng hash an toàn (Argon2id), không lưu plaintext.

• Phân quyền ở service/backend layer, không chỉ ẩn nút trên giao diện.

• Ghi audit log cho đăng nhập, thay đổi cấu hình/model, thay đổi quyền và thao tác xoá dữ liệu.

6. Lộ trình triển khai đề xuất

Giai đoạn	Kết quả bàn giao	Ước tính
1. UI Foundation	MainWindow, sidebar, header, dark theme, điều hướng các trang.	1–2 ngày
2. Camera + YOLO	Live camera, ONNX inference, bbox, class/confidence, FPS/latency.	2–3 ngày
3. Dashboard	Thẻ thống kê và trạng thái hệ thống cập nhật từ dữ liệu thật.	1–2 ngày
4. Database + History	SQLite, lưu kết quả, lịch sử, lọc và xuất báo cáo.	2–3 ngày
5. Settings + Users	Cấu hình, login, roles và permissions.	2–3 ngày
6. Reliability + Deploy	Logging, recovery, kiểm thử và đóng gói Windows/Linux.	2–4 ngày

Ước tính tổng thể: khoảng 10–17 ngày làm việc tập trung cho phiên bản tương đối hoàn chỉnh. Đây là ước lượng lập kế hoạch, không phải cam kết tiến độ; thời gian có thể thay đổi theo lỗi tích hợp, phần cứng và mức độ hoàn thiện mong muốn.

7. Tiêu chí nghiệm thu MVP

☐ Ứng dụng khởi chạy được trên Windows và hiển thị đúng dark theme.

☐ Sidebar chuyển được giữa Dashboard, Live Inspection và History.

☐ Camera hiển thị realtime, không làm treo UI; Start/Stop hoạt động ổn định.

☐ YOLO ONNX phát hiện được Fresh/Rotten; bounding box và confidence hiển thị đúng.

☐ Có FPS và inference latency; trạng thái camera/model được cập nhật.

☐ Kết quả inspection được lưu và có thể truy xuất lại.

☐ Có xử lý lỗi cơ bản, giải phóng tài nguyên khi đóng ứng dụng.

8. Phạm vi chưa cần làm ngay

• Không cần xây dựng web frontend hoặc học React/JavaScript cho phiên bản desktop này.

• Không cần triển khai FastAPI ngay; chỉ bổ sung khi cần kết nối từ xa, robot/PLC hoặc nhiều client.

• Không cần làm hệ thống plugin động phức tạp ở giai đoạn đầu.

• Không cần PostgreSQL nếu ứng dụng chỉ chạy độc lập trên một máy; có thể nâng cấp khi cần nhiều máy dùng chung dữ liệu.

• Không thay đổi thuật toán/model YOLO đang hoạt động chỉ để phục vụ việc dựng giao diện.

Nguyên tắc triển khai: ưu tiên một MVP chạy ổn định trước. Mỗi giai đoạn phải chạy thử, kiểm tra và giữ lại phiên bản ổn định trước khi mở rộng. Frontend là lớp vận hành và hiển thị; backend camera/AI là lớp xử lý chuyên môn.